

IE7615 Deep Learning for AI

Comparing Convolutional Neural Networks for 73-Class Object Recognition

Discriminative Project: Milestone 1 Report

Group 7

Tanmayi Shurpali
Santhoshkumar Chandrasekar
Praniti Sunil Kale
Mukul Sridar Haladhi

Instructor: Xuemin Jin
Submission date: October 4, 2026

Abstract

This report documents Milestone 1 of the discriminative project: a closed-set image classifier that recognises which of 73 registered everyday objects appears in a photograph taken by a student. We audited and cleaned the final merged course dataset, built one fixed stratified split, and trained five convolutional neural networks under an identical recipe: a custom CNN trained from scratch and four ImageNet-pretrained networks (ResNet-50, EfficientNet-B0, ConvNeXt-Tiny and DenseNet-121).

Transfer learning proved decisive. The pretrained models reached 98.66% to 99.23% test accuracy, roughly ten points above the scratch CNN at 89.06%. On clean photos the four pretrained models are statistically tied, so we also tested every model on deliberately degraded images (small, cropped, occluded and low-quality photos). There the models separated clearly. **ConvNeXt-Tiny is the best model for this dataset**: it was selected using validation data only and achieved 99.23% test accuracy (95% CI 98.7% to 99.7%) with 8 errors on 1,042 unseen photos, and it was the most robust model under every simulated condition.

Contents

1. Introduction and Problem Overview	
2. Dataset	
3. Methodology	
4. Models Tested	
5. Experimental Results and Performance of Each Model	
6. Comparison, Analysis and Discussion	9
7. Final Model Selection and Justification	12
8. Limitations and Future Work	13
9. Conclusion	13
Appendix A. Supplementary Experiments, Code and Reproducibility	14

1. Introduction and Problem Overview

Deep learning has become the standard approach to image recognition, and convolutional neural networks (CNNs) in particular learn hierarchical visual features directly from pixels. This milestone applies CNNs to a custom object-recognition task. Each input is one RGB photograph, and the model must output exactly one of 73 Object IDs (OBJ001 to OBJ073). Each ID refers to one physical object registered by a group of students, for example a pair of sunglasses, a tumbler, a backpack or a plush toy. The set is closed: shown an object outside the list, the classifier still answers with the most similar of the 73 IDs.

The goals of this milestone are to (1) audit, clean and split the dataset reproducibly, (2) train and compare a from-scratch CNN against pretrained architectures under identical conditions, (3) evaluate the models on accuracy, robustness, efficiency and statistical significance, and (4) select the model best suited to this dataset using validation data only.

1.1 Why this dataset is challenging

- **Few images per class.** After cleaning there are 89 to 106 photos per object, about 66 training images per class, which is very little for a network trained from scratch.
- **Small, distant objects.** Many photos show the object far away, for example a remote on a large bed.
- **Partial views and occlusion.** Some objects are cut off at the frame edge or partly covered.
- **Uneven photo quality.** Lighting, focus, resolution and camera differ between students.
- **Multi-object scenes and look-alike classes.** Some photos contain several registered objects, and some classes look alike (two different pairs of dark sunglasses).
- **Background correlation.** Each object was photographed in its owner's rooms, so the background itself carries information about the label.

1.2 How performance is measured

Accuracy is the share of test photos classified correctly. **Macro-F1** is the F1 score computed per class and averaged with equal weight, so every object counts equally; it is our primary selection metric. Supporting measures are 95% bootstrap confidence intervals, accuracy on a hard subset without near-duplicates, top-5 accuracy, accuracy under four image perturbations, McNemar significance tests, inference speed and model size.

2. Dataset

We used the final merged dataset released by the teaching assistants: one folder per Object ID holding 7,343 image files in total, at the TA standard resolution of 224×224 pixels. Every cleaning step was carried out in code, in a fixed order, and no file in the shared dataset was modified. Labels always come from the folder name, never the filename.

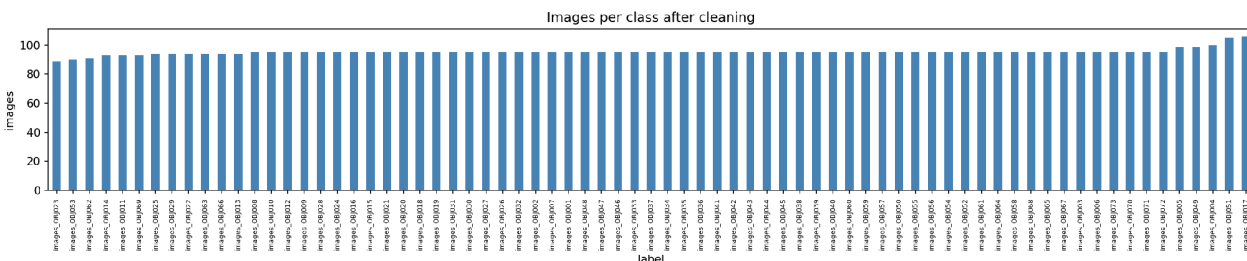


Figure 1. Images per class after cleaning. Classes are balanced: 89 to 106 images each (largest-to-smallest ratio 1.19).

Table 1. Data cleaning steps, in the order they were run.

Step	What was done	Result	Why it matters
1. Index & junk	Located the 73 class folders; skipped hidden files such as .DS_Store	7,343 images; 2 junk entries skipped	Junk would crash training
2. Validation	Opened every image; recorded size and mode	0 corrupt images	A broken file can stop a run
3. Filename check	Compared filename ID with folder	Reported only; nothing relabelled	Folder is ground truth
4. Background removal	Removed files tagged _bg_ or BG###	385 removed (kept aside)	Empty scenes teach rooms, not objects
5. Duplicates	MD5 fingerprint; one copy kept	16 removed, 0 cross-class conflicts	Prevents train/test leakage
6. Size & colour	Read as RGB at 224×224, no centre crop	All inputs 224×224×3	Objects near edges are never cut away
7. Final set	Object photos for training and evaluation	6,942 images; 89–106 per class	Balanced, leakage-free

One class, OBJ004, does not tag its background photos, so they could not be identified automatically and remain in the data (a limitation). One image in OBJ054 is a phone screenshot showing personal details; because the dataset is locked it stayed in the data, but it is automatically blurred in every figure. As an extra precaution, 12 images that an automatic CLIP-based screen flagged as possibly showing a phone screen are also blurred in every figure; like the rest of the locked dataset, they remain in the data.

2.1 Train, validation and test split

A stratified 70 / 15 / 15 split with seed 42 was used, so every Object ID appears in all three sets in the same proportion. The file list was sorted before splitting, making the split identical on every run, and all five models used exactly this split.

Table 2. The data split.

Set	Images	Per class (approx.)	Used for
Training	4,859	62 to 74	Learning the model weights
Validation	1,041	13 to 16	Early stopping, best epoch, final model selection
Test	1,042	13 to 16	Final, unbiased reporting only; never used for any decision

2.2 Data quality checks

Two diagnostics used features from a frozen ImageNet ConvNeXt-Tiny and changed no data. **Label audit:** a logistic-regression classifier scored with 5-fold cross-validation reached 97.9% out-of-fold accuracy, confirming the labels are consistent. Fifty suspect images that confidently resemble another ID are almost all multi-object scenes or look-alike objects, not wrong labels (Figure 2). **Near-duplicate check:** students often took bursts of near-identical photos. 99 test photos have a near-twin in training (cosine similarity ≥ 0.92); the remaining 943 form the **hard subset**, on which accuracy is reported separately.

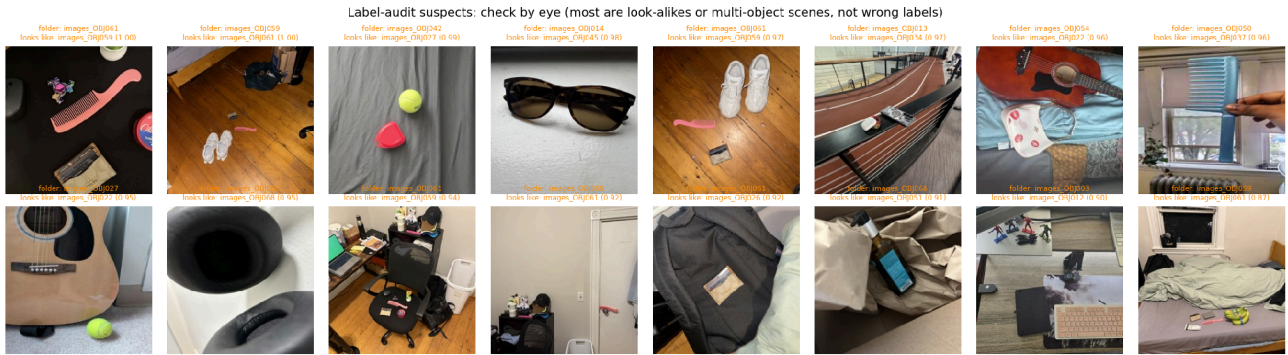


Figure 2. Label-audit suspects: the folder label versus the class the image most resembles. Most are multi-object scenes or look-alikes.

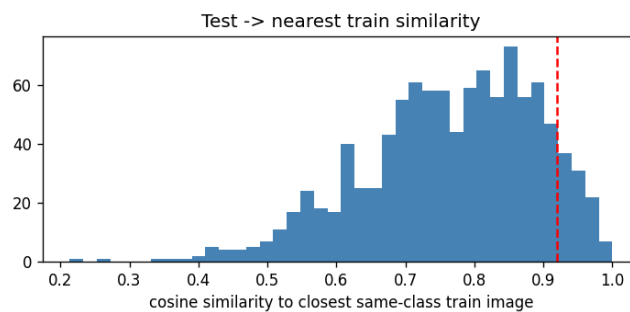


Figure 3. Similarity of each test photo to its closest same-class training photo (threshold 0.92).

3. Methodology

3.1 Preprocessing and augmentation

Images are decoded with camera orientation applied, converted to RGB and kept at 224×224 without centre cropping. Pixels are normalised with the ImageNet mean and standard deviation for all models, including the custom CNN. Training images receive random augmentation each epoch, designed to simulate the specific problems of student photos; validation and test images are never augmented.

Table 3. Training augmentation.

Transform	Setting	Problem it simulates
Random resized crop	Keep 50–100% of image, aspect 0.8–1.25	Partial views, different framing
Zoom-out	$p = 0.25$; shrink to 45–85% on a blurred copy	Object far away and small
Horizontal flip	$p = 0.5$	Either side viewpoint
TrivialAugment Wide	One random colour / contrast / sharpness / geometric change	Lighting, camera, quality
Random erasing	$p = 0.25$; 2–15% of area filled	Object partly covered

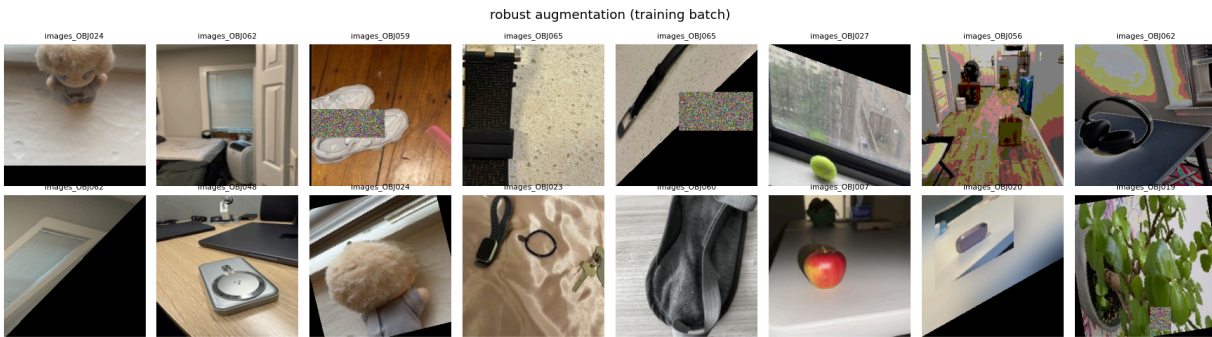


Figure 4. A training batch with the robust augmentation actually used (zoom-out, colour changes, erased patches).

3.2 Shared training recipe

All models share one recipe so that differences in results reflect the architectures, not different tricks: **AdamW** with no weight decay on biases and norm layers; the new output layer learns 3 to 10 times faster than pretrained layers; **linear warm-up then cosine decay** updated every step; **cross-entropy with label smoothing 0.1** and class weights (the 1.19 class ratio triggers the weighted loss); mixed precision and gradient clipping at 1.0; the checkpoint is the **epoch with the best validation macro-F1**; early stopping on validation macro-F1; seed 42 with a checkpoint every epoch so a disconnected run resumes identically.

3.3 Evaluation protocol

- The validation set drives early stopping and final model selection. The test set is used once, only to report results.
- 95% confidence intervals come from 2,000 bootstrap resamples of the 1,042 test photos.
- **McNemar's exact test** compares two models on the same photos, counting only images where exactly one model is correct.
- **Robustness tests** re-evaluate every photo under four perturbations with a fixed seed per image, so all models see identical inputs: small object (half size on a blurred canvas, about 2× farther), cropped (random 70% window), occluded (grey box over 25% of the area) and low quality (4× downscale, brightness × 0.6, JPEG quality 20).
- **Grad-CAM** heatmaps show which regions drive a prediction, and a **background-shortcut test** runs each model on 378 background-only photos that contain no object.



Figure 5. The four simulated photo problems applied to one test image.

4. Models Tested

Before the final run, two group members ran exploratory experiments with different architectures on slightly different cleaning and splits. Those numbers are not comparable with this report and were used only to shortlist the four strongest pretrained architectures, which were then retrained from the start on the common cleaned

data and split together with the best custom CNN as a baseline. Figures label the models by experiment ID: A2, C, D, E and G.

Table 4. The five models compared.

ID	Model	Training	Params	Defining idea
A2	Custom CNN	From scratch	3.41 M	VGG-style: five stages of paired 3×3 convs + batch norm + max-pool (224→7 px; 32→384 channels), global average pooling, dropout 0.3, Kaiming init
C	ResNet-50	ImageNet, fine-tuned	23.66 M	Residual blocks with skip connections make very deep networks trainable
D	EfficientNet-B0	ImageNet, fine-tuned	4.10 M	Mobile inverted bottlenecks with squeeze-and-excitation; depth, width and resolution scaled together
E	ConvNeXt-Tiny	ImageNet, fine-tuned	27.88 M	Modernised pure CNN borrowing ViT ideas: 7×7 depthwise kernels, layer norm, inverted bottlenecks
G	DenseNet-121	ImageNet, fine-tuned	7.03 M	Each layer receives feature maps of all earlier layers; compact feature reuse

For the four pretrained models only the final layer is replaced by a new 73-class layer, and all layers are then fine-tuned. The scratch model gets three times more epochs and a longer warm-up because it starts from random weights.

Table 5. Per-model training settings.

Model	Max epochs	Warm-up	Patience	Batch	LR new / pretrained	Weight decay	Epochs run (best)
A2 Custom CNN	60	3	15	32	0.002 / 0.002	0.05	60 (58)
C ResNet-50	20	1	6	32	0.001 / 0.0001	0.01	18 (12)
D EfficientNet-B0	20	1	6	32	0.001 / 0.0003	0.01	20 (20)
E ConvNeXt-Tiny	20	1	6	32	0.001 / 0.0001	0.05	20 (20)
G DenseNet-121	20	1	6	32	0.001 / 0.0001	0.01	20 (18)

5. Experimental Results and Performance of Each Model

5.1 Overall performance

Table 6 is the central result. Every number comes from the same 1,042 test photos, which no model saw during training or selection.

Table 6. Performance on the test set (percentages unless noted). Inference time per image at batch size 32.

Model	Val F1	Test acc. (95% CI)	Test F1	Hard subset	Top-5	Errors	Params	Train time	ms / img
A2 Custom CNN	89.56	89.06 (87.1–91.1)	88.72	87.91	97.12	114	3.41 M	30.6 min	0.77
C ResNet-50	99.22	98.66 (98.0–99.3)	98.65	98.52	99.71	14	23.66 M	11.4 min	1.26
D EfficientNet-B0	99.41	98.75 (98.1–99.3)	98.74	98.62	99.81	13	4.10 M	11.7 min	0.70
E ConvNeXt-Tiny	99.61	99.23 (98.7–99.7)	99.23	99.15	99.81	8	27.88 M	13.4 min	1.43
G DenseNet-121	99.13	98.66 (97.9–99.3)	98.63	98.52	99.90	14	7.03 M	14.1 min	1.51

The four pretrained models all exceed 98.6% on every clean metric, while the scratch CNN stays near 89%. ConvNeXt-Tiny has the best validation macro-F1, test accuracy, test macro-F1, hard-subset accuracy and the fewest errors. EfficientNet-B0 is the fastest and second-smallest model and is second on almost every metric. Removing the 99 near-duplicate test photos lowers pretrained-model accuracy by only 0.08 to 0.14 points (E:

99.23% to 99.15%), so the results are not inflated by burst shots. Test-time augmentation changes the pretrained models by at most 0.19 points, so we report results without it.

5.2 Training behaviour

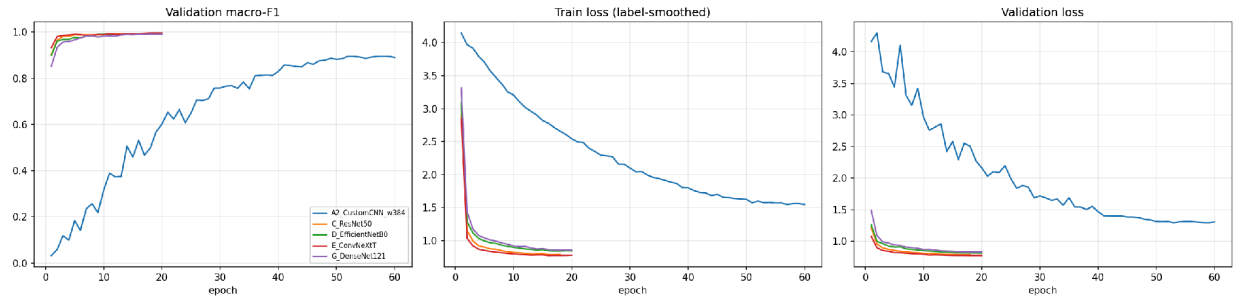


Figure 6. Validation macro-F1, training loss and validation loss per epoch for all five models.

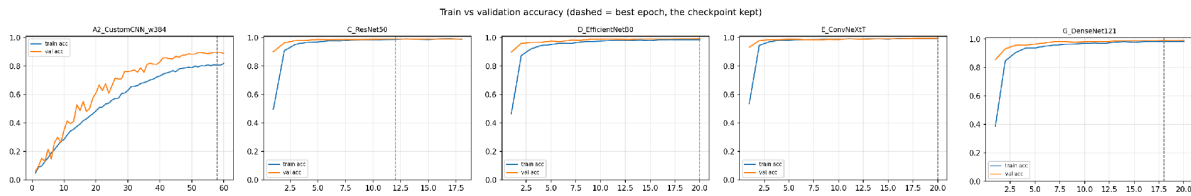


Figure 6b. Training versus validation accuracy per model (dashed line: best epoch, the checkpoint kept).

All pretrained models exceed 85% validation accuracy after the first epoch (E 93.4%, C 90.3%, D 90.0%, G 85.6%) and pass 98% within two to seven epochs. ResNet-50 stopped early at epoch 18, while EfficientNet-B0 and ConvNeXT-Tiny reached their best validation score in the final epoch, so a slightly longer schedule might have improved them a little further. The scratch CNN starts at 6% (chance is 1.4%) and climbs for all 60 epochs, reaching its best at epoch 58; it is under-trained rather than saturated. Validation loss never trends upward for any model, so there is no sign of overfitting. For the scratch model, validation accuracy sits above training accuracy because training accuracy is measured on heavily augmented images with dropout active.

5.3 Robustness to real-world photo problems

Table 7. Test accuracy (%) under each simulated photo problem.

Model	Clean	Small object	Cropped	Occluded	Low quality	Mean of four
A2 Custom CNN	89.06	76.68	90.12	74.95	26.10	66.96
C ResNet-50	98.66	95.20	98.08	89.64	32.92	78.96
D EfficientNet-B0	98.75	94.82	97.98	89.44	45.01	81.81
E ConvNeXT-Tiny	99.23	97.60	98.46	91.17	51.73	84.74
G DenseNet-121	98.66	93.47	98.18	82.82	19.87	73.59

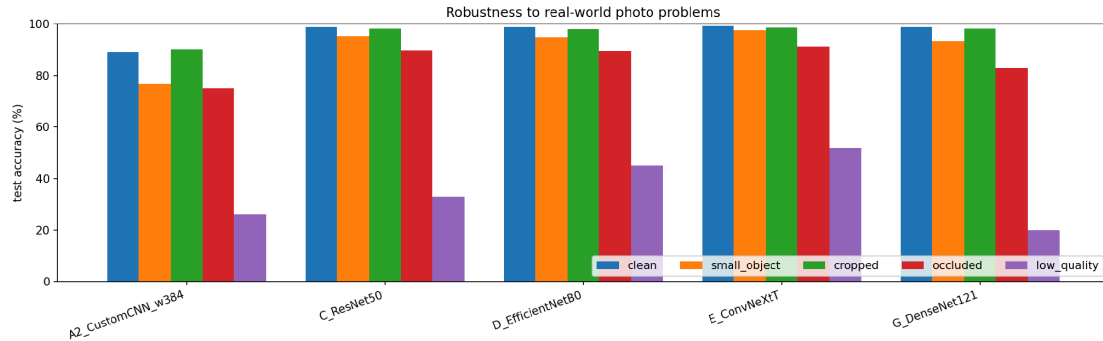


Figure 7. Test accuracy of each model on clean photos and under each perturbation.

Cropping costs the pretrained models less than one point, so partial views are handled well. Small objects and occlusion cost more, and low-quality photos are by far the hardest condition for every model (20% to 52%). ConvNeXt-Tiny is best in every condition and has the smallest drop from clean accuracy (14.5 points). DenseNet-121 ties the others on clean photos yet is the most fragile once conditions worsen (19.9% on low-quality photos), showing that clean accuracy alone hides large differences.

6. Comparison, Analysis and Discussion

6.1 Transfer learning is the decisive factor

Every pretrained model beats the custom CNN by at least 9.6 points of test accuracy and makes 8 to 14 errors instead of 114. With about 66 training photos per class, a network trained from scratch must discover edges, textures and shapes from very little data, while pretrained networks already learned these building blocks from 1.2 million ImageNet photos and only need to adapt them. The scratch model also trained the longest (30.6 minutes) and was still the weakest.

6.2 Statistical significance

Table 8. McNemar exact tests between the selected model and every other model.

Compared ConvNeXt-Tiny	with	E	Only E correct	Only other correct	p-value	Significant ($p < 0.05$)?
A2 Custom CNN		110	4		< 0.001	Yes
C ResNet-50		11	5		0.210	No
D EfficientNet-B0		10	5		0.302	No
G DenseNet-121		9	3		0.146	No

Only the gap to the scratch CNN is significant. Among the pretrained models ConvNeXt-Tiny wins more disagreements than it loses, but with so few disagreements this could be chance, and the confidence intervals overlap. On clean photos the four pretrained models are statistically tied, so the case for ConvNeXt-Tiny rests on robustness (Section 5.3), not on a significant clean-accuracy gain.

6.3 Error analysis of the selected model

ConvNeXt-Tiny makes 8 errors on test (4 on validation) and classifies 60 of 73 classes perfectly; macro precision, recall and F1 are all 99.2%. The confusion matrix is almost purely diagonal (Figure 8). Every error has a visible cause (Table 9, Figure 9): multi-object scenes (4), look-alike objects (2), a very small object (1) and a phone screenshot that is not an object photo (1).

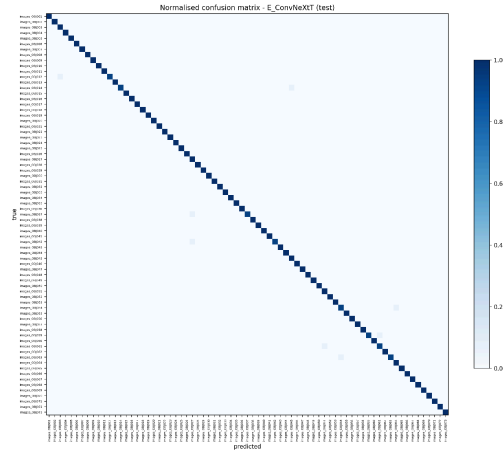


Figure 8. Normalised confusion matrix of the selected model (E, ConvNeXt-Tiny) on the test set.

Table 9. All 8 test errors of the selected model, sorted by confidence.

True ID	Predicted	Confidence	Likely cause
OBJ059	OBJ061	0.66	Multi-object room scene; same items appear in both classes' photos
OBJ061	OBJ051	0.49	Several objects together (bananas and other items)
OBJ037	OBJ027	0.47	Object very small and far away in a large hall
OBJ042	OBJ027	0.43	Multi-object scene; OBJ027 object clearly visible next to it
OBJ014	OBJ045	0.41	Look-alike class: both are dark sunglasses
OBJ012	OBJ003	0.40	Look-alike: dark upright rectangular device against a window
OBJ063	OBJ054	0.16	Cluttered table with several items
OBJ054	OBJ064	0.13	Not an object photo: the private phone screenshot

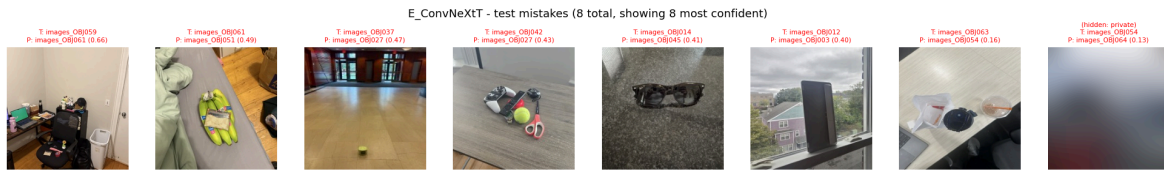


Figure 9. The 8 test errors of ConvNeXt-Tiny (T: true ID, P: predicted ID, with confidence). The screenshot is blurred.

All eight errors have confidence below 0.67 while the model's average test confidence is 0.89, so the model is uncertain exactly when it is wrong; a confidence threshold could flag such photos for review.

6.4 Photos that every model finds hard

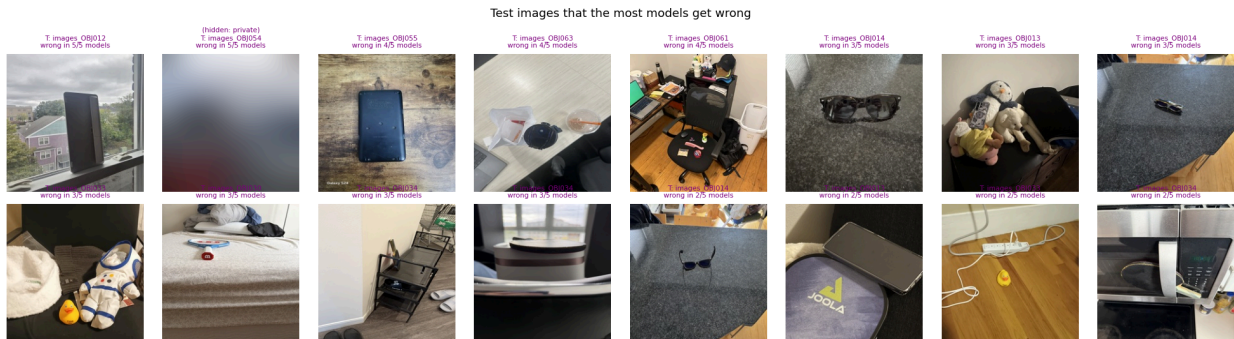


Figure 10. Test photos that the most models get wrong (out of 5).

The hardest photos are shared across architectures: the dark upright device of OBJ012 and the OBJ054 screenshot are wrong in all five models, and the multi-object scenes of OBJ055, OBJ061 and OBJ063 in four. These are data-level difficulties, not weaknesses of one architecture, and they match the label-audit suspects.

6.5 Grad-CAM: where the models look

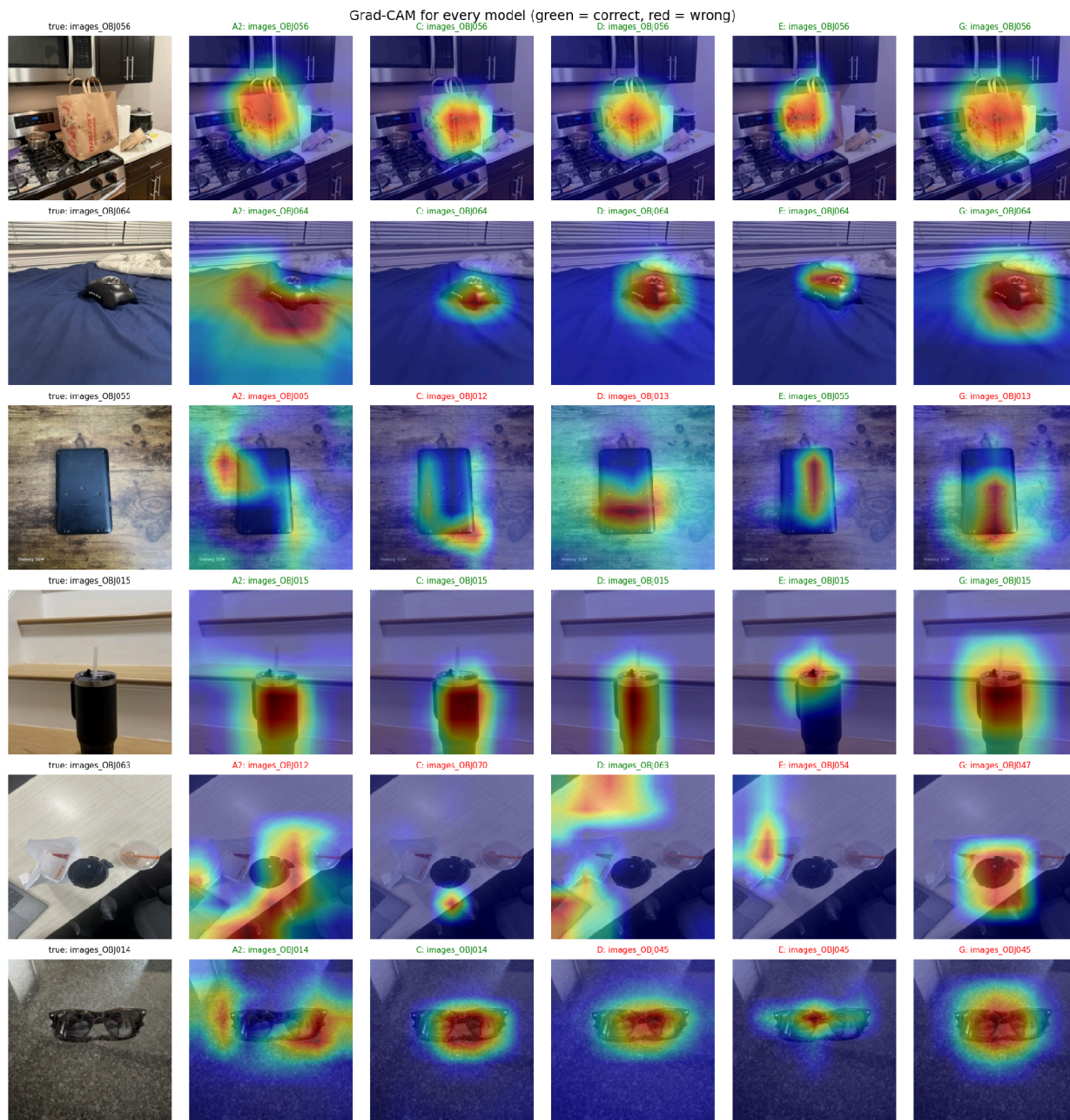


Figure 11. Grad-CAM heatmaps for the same six test photos across all five models (green title = correct, red = wrong).

For clearly visible objects (a shopping bag, a device on a bed, a tumbler) all models are correct and the pretrained networks focus tightly on the object, while the scratch model's heat is wider and spills onto the bed and surroundings, which helps explain its lower accuracy. For OBJ055 only ConvNeXt-Tiny is correct, with heat along the device itself. The OBJ014 sunglasses are confused with OBJ045 by three models, yet the heat is still on the sunglasses: these are look-alike confusions, not background shortcuts.

6.6 Background-shortcut test

Table 10. Predictions on 378 background-only photos (no object present).

Model	Own ID predicted	Chance	Conf. on empty scenes	Conf. on real photos
A2 Custom CNN	33.1%	1.37%	0.377	0.799
C ResNet-50	36.5%	1.37%	0.346	0.867
D EfficientNet-B0	38.1%	1.37%	0.376	0.886
E ConvNeXt-Tiny	35.7%	1.37%	0.356	0.890
G DenseNet-121	34.7%	1.37%	0.332	0.867

Every model predicts the photographer's own Object ID for about one third of empty scenes, roughly 25 times chance, so the room does hint at the label. However confidence on these scenes is low (0.33 to 0.38) compared with real photos (0.80 to 0.89), and Grad-CAM puts the heat on the object when it is present. The room is therefore a weak secondary cue rather than the main signal. This correlation is built into how the data was collected and cannot be removed by cleaning; excluding the background photos from training was the best mitigation available.

6.7 Summary of the comparison

- **Pretrained vs scratch:** a large, significant gap (about 10 points of accuracy; 8–14 errors versus 114).
- **Among pretrained models:** a statistical tie on clean photos (98.66% to 99.23%).
- **Robustness:** a clear ranking, E (84.7) ahead of D (81.8), C (79.0) and G (73.6).
- **Efficiency:** D is the smallest and fastest pretrained model; E is the largest but still takes only 1.43 ms per image.
- **Errors:** concentrated in multi-object scenes and look-alike classes, shared by all models, and made with low confidence.

7. Final Model Selection and Justification

To keep the reported test results unbiased, the model was chosen with validation data only, using a rule fixed before looking at any test result: **(1)** keep every model within 0.5 points of the best validation macro-F1; **(2)** among those, choose the highest mean accuracy on perturbed validation images; **(3)** if still tied, choose the fewest parameters. Near 99%, clean validation scores differ by only a few images, so robustness is added as the second criterion.

Table 11. The selection rule applied to validation data.

Model	Val macro-F1	Within 0.5 of best?	Mean acc. on perturbed val.	Parameters
E ConvNeXt-Tiny	99.61	Yes (best)	83.98	27.88 M
D EfficientNet-B0	99.41	Yes	81.63	4.10 M
C ResNet-50	99.22	Yes	79.04	23.66 M
G DenseNet-121	99.13	Yes	73.87	7.03 M
A2 Custom CNN	89.56	No	65.68	3.41 M

Step 1 keeps C, D, E and G. Step 2 picks ConvNeXt-Tiny with 83.98%; step 3 is not needed.

7.1 Selected model: ConvNeXt-Tiny (E), fully fine-tuned

- **Highest accuracy:** 99.23% test accuracy (95% CI 98.7–99.7%), 99.23 macro-F1, 99.15% on the hard subset, top-5 99.81%, only 8 errors in 1,042 unseen photos.
- **Strongest robustness:** best in every perturbation on both validation and test, with the smallest drop from clean accuracy. For a dataset full of small, partly hidden and poorly lit objects this is the deciding property.
- **Confirmed on held-out data:** chosen on validation, then leading again on the untouched test set.
- **Uncertain when wrong and object-focused:** all errors have confidence below 0.67, and Grad-CAM shows tight focus on the object.
- **Practical cost:** 13.4 minutes of training and 1.43 ms per image.

7.2 Trade-offs

On clean photos ConvNeXt-Tiny is not significantly better than ResNet-50, EfficientNet-B0 or DenseNet-121; its advantage is robustness. It is also the largest model tested. If memory or speed were the priority, **EfficientNet-B0** is the natural alternative: 6.8 times smaller and twice as fast, at the cost of 0.48 points of clean accuracy, 5 more test errors and about 2.9 points less robustness. A rule that broke ties by size alone would have picked EfficientNet-B0; we chose robustness as the tie-breaker before seeing test results because the main difficulty of this dataset is far-away, partly hidden and low-quality photos.

8. Limitations and Future Work

8.1 Limitations

- **Background reliance:** background is correlated with the label throughout the dataset (Section 6.6).
- **Undetected background photos:** OBJ004 has no background tags, so such shots remain in the data.
- **Same environments in train and test:** accuracy in a completely new environment would likely be lower.
- **Low-quality photos:** all models lose more than half their accuracy (best 51.7%).
- **Multi-object scenes:** a single label is arguably ambiguous for some photos.
- **One split and a closed set:** results come from one fixed split, and the classifier cannot answer “unknown”.

8.2 Future work

- Add blur, downscaling and JPEG compression to training augmentation to target the low-quality weakness.
- Use background-swap or object cut-and-paste augmentation and a test set with new backgrounds to reduce and measure background reliance.
- Repeat the experiments with several random splits and report the mean and spread.
- Calibrate confidence and add an “unknown” option by thresholding; train the scratch model longer and try a small vision transformer.

9. Conclusion

We built a clean, reproducible pipeline for recognising 73 student-registered objects and compared a custom CNN trained from scratch with four ImageNet-pretrained networks under identical conditions. Transfer learning is clearly the right approach for a dataset of this size, lifting test accuracy from 89% to between 98.7% and 99.2%. Because the four pretrained networks are statistically tied on clean photos, we selected the final model on robustness to realistic photo problems, using validation data only. **ConvNeXt-Tiny, fully fine-tuned,**

is the best model for this dataset: 99.23% test accuracy and macro-F1, 8 errors on 1,042 unseen photos, the best robustness of all models, and low confidence exactly on the photos it gets wrong. Its remaining weaknesses, low-quality photos and reliance on familiar backgrounds, define our next steps.

Appendix A. Supplementary Experiments, Code and Reproducibility

A.1 Additional experiments. Unlike the exploratory runs mentioned in Section 4, the two experiments below used the same cleaned dataset and split as the main study (4,859 / 1,041 / 1,042). Each was run independently by an individual member with their own pipeline. Their training setups differ from the main study and they played no part in model selection, but they reinforce the main findings.

Table A1. Six additional pretrained architectures (Apple-silicon GPU; times not comparable with Table 6).

Additional architecture (year)	Test acc.	Macro-F1	Errors	Params	Train time
ResNet-101 (2015)	98.4%	0.983	17	42.7 M	25.6 min
ShuffleNetV2 $\times 1.5$ (2018)	98.0%	0.980	21	2.6 M	7.1 min
GoogLeNet (2014)	97.4%	0.974	27	5.7 M	15.0 min
MNASNet-1.0 (2019)	96.2%	0.961	40	3.2 M	11.5 min
SqueezeNet 1.1 (2016)	95.4%	0.954	48	0.8 M	4.4 min
AlexNet (2012)	94.2%	0.941	60	57.3 M	6.4 min

None surpasses ConvNeXt-Tiny's 99.23%. Design matters more than size: AlexNet has the most parameters and the lowest accuracy, ResNet-101 does not improve on ResNet-50, and ShuffleNetV2 reaches 98.0% with only 2.6 M parameters.

Table A2. Independent replication with augmentation ablation.

Independent replication (six models)	Test acc.	Macro-F1	Params
Custom CNN (from scratch)	88.6%	0.883	3.4 M
ConvNeXt-Tiny, frozen backbone (last layer only)	97.4%	0.974	27.9 M
ResNet-50	98.7%	0.986	23.7 M
EfficientNet-B0	98.2%	0.982	4.1 M
ConvNeXt-Tiny	99.4%	0.994	27.9 M
ConvNeXt-Tiny, basic augmentation only (ablation)	98.7%	0.987	27.9 M

The replication again selected ConvNeXt-Tiny on validation data (99.4% test accuracy). The ablation shows the value of the targeted augmentation: versus crop-and-flip only, it improved accuracy by 20.4 points on low-quality photos (57.0% vs 36.7%), 11.3 points on small objects (97.9% vs 86.6%) and 3.2 points on occluded objects, and clean accuracy improved significantly ($p = 0.02$).

A.2 Misclassified images. The notebook saves a grid of up to 16 mistakes per model and split (most confident first), plus a CSV listing every error (misclassified_all_models.csv). The selected model's test errors are shown in Figure 9; the grids for all other models are in the repository's figures folder.

A.3 GitHub repository, code and reproducibility. All code, scripts and artifacts generated for Milestone 1 are available in our GitHub repository. This includes the training scripts, metrics logs, plots, comparison utilities and dataset extraction workflows. The repository contains all the code needed to reproduce the results and serves as a central location for tracking progress across all project milestones.

GitHub Link: [GitHub](#)